

Получение и анализ дампов приложений СБИС в Linux

Многие из разработчиков сталкиваются с необходимостью анализа [дампов](#) при работе с сервисами на платформе СБИС под Linux, в которых используются компоненты на C++. В данной статье приведено описание решения, которое позволяет упростить работу с дампами.

Более подробное представление о том, что такое дамп и как с ним работать приведено [здесь](#).

Получение дампов "боевого" приложения

Как правило, путь к полному дампу прикладываются к ошибке (см. [пример](#)), которую получает разработчик на портале СБИС.

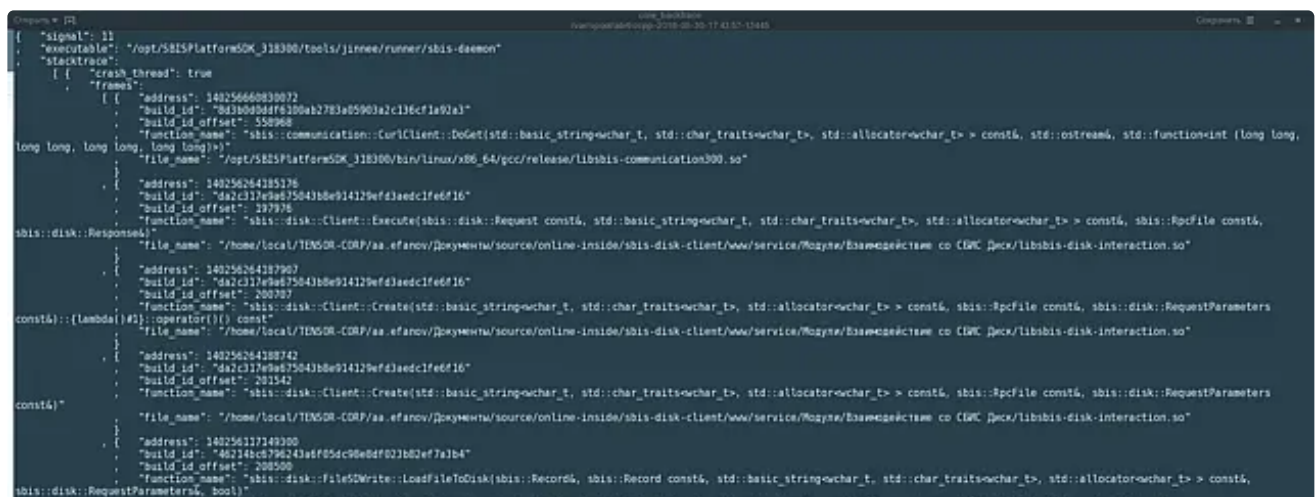
Получение дампов для локального стенда

При "падении" воркера локального стенда дамп памяти сохраняется на рабочем компьютере разработчика. Записью и сохранением дампов занимается [abrt](#) (automatic bug reporting tool), настройка которого предварительно выполнена на рабочих компьютерах сотрудников разработки в компании "Тензор". abrt - стандартная линуксовая утилита для записи дампов и она НЕ СОЗДАЕТ zip-архив, аналогичный тем, которые генерируются на серверах БЛ.

Дамп локального стенда находится в директории:

```
/var/spool/abrt/ccpp*/coredump
```

Для анализа дампа могут пригодиться дополнительные файлы. Например, [core_backtrace](#) с содержимым стека вызовов в json-формате.



Такие файлы находятся в директории:

```
/var/spool/abrt/ccpp*/
```

Если генерируемых abrt-файлов недостаточно для диагностики проблемы, используйте [отладчик GDB](#) для анализа дампов.

Анализ дампов при помощи утилиты analyse_dump

Для автоматизации анализа дампов была написана утилита **analyse_dump**. Она входит в штатную поставку образа виртуальной машины, рекомендованной платформой для использования в качестве [рабочего места разработчика в CentOS](#):

```
svc-fs.corp.tensor.ru/public_software/Программное%20обеспечение/  
Разработка/CentOS7
```

Также утилита устанавливается из [репозитория SBIS-development](#):

```
sudo yum install sbis-analyse-dump
```

Утилита имеет два режима работы: [интерактивный](#) и [неинтерактивный](#).

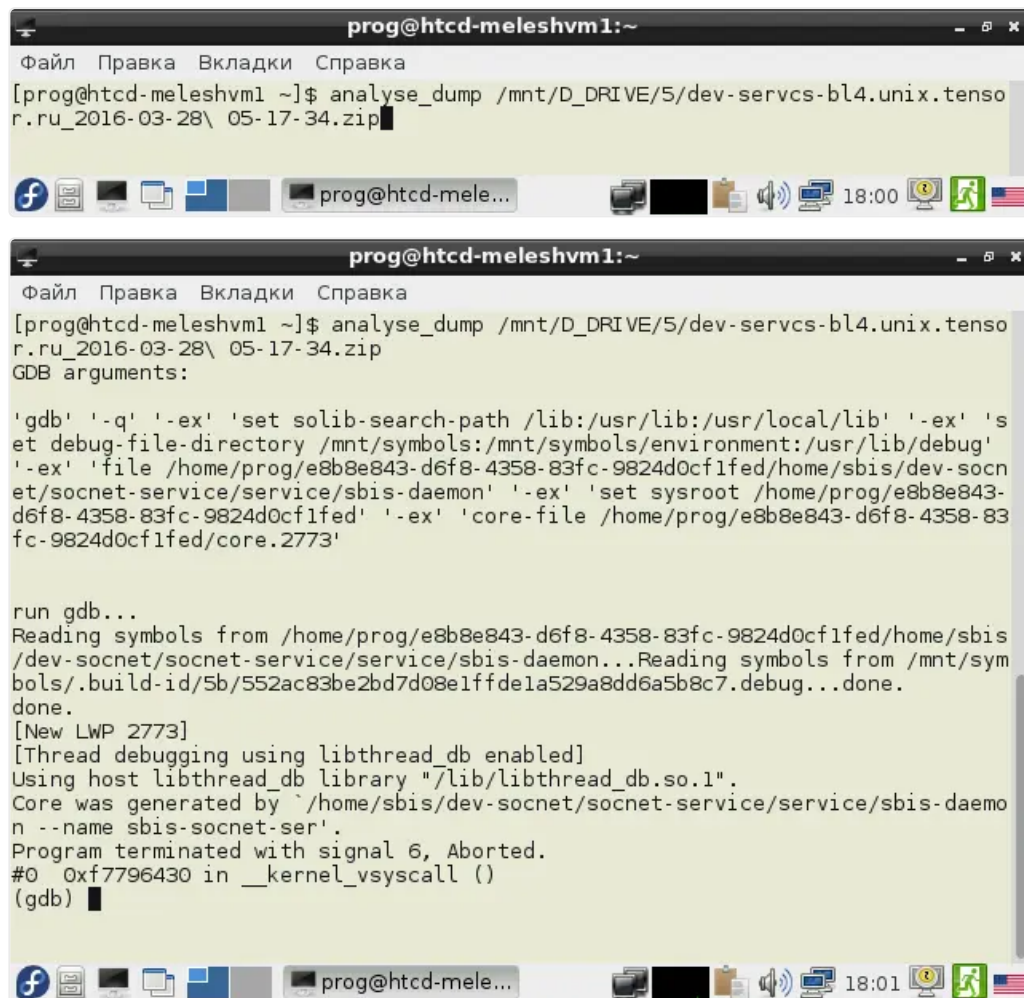
Интерактивный режим analyse_dump. Загрузка дампа в GDB

В интерактивном режиме утилита запускает должным образом настроенный отладчик GDB с правильно загруженным дампом. Для этого в консоли выполняем команду:

```
analyse_dump <путь к zip-файлу дампа>
```

Далее можно производить отладку при помощи команд GDB. Информацию по отладчику можно найти на [официальном сайте](#).

В приведенном ниже примере производится загрузка дампа "/mnt/D_DRIVE/5/dev-servcs-bl4.unix.tensor.ru_2016-03-28_05-17-34.zip". Скрипт настроит окружение для анализа дампа и запустит интерактивную консоль отладчика GDB.



```
prog@htcd-meleshvm1:~  
Файл Правка Вкладки Справка  
[prog@htcd-meleshvm1 ~]$ analyse_dump /mnt/D_DRIVE/5/dev-servcs-bl4.unix.tensor.ru_2016-03-28_05-17-34.zip  
prog@htcd-meleshvm1:~  
Файл Правка Вкладки Справка  
[prog@htcd-meleshvm1 ~]$ analyse_dump /mnt/D_DRIVE/5/dev-servcs-bl4.unix.tensor.ru_2016-03-28_05-17-34.zip  
GDB arguments:  
  
'gdb' '-q' '-ex' 'set solib-search-path /lib:/usr/lib:/usr/local/lib' '-ex' 'set debug-file-directory /mnt/symbols:/mnt/symbols/environment:/usr/lib/debug' '-ex' 'file /home/prog/e8b8e843-d6f8-4358-83fc-9824d0c1fed/home/sbis/dev-socnet/socnet-service/service/sbis-daemon' '-ex' 'set sysroot /home/prog/e8b8e843-d6f8-4358-83fc-9824d0c1fed' '-ex' 'core-file /home/prog/e8b8e843-d6f8-4358-83fc-9824d0c1fed/core.2773'  
  
run gdb...  
Reading symbols from /home/prog/e8b8e843-d6f8-4358-83fc-9824d0c1fed/home/sbis/dev-socnet/socnet-service/service/sbis-daemon...Reading symbols from /mnt/symbols/.build-id/5b/552ac83be2bd7d08e1ffde1a529a8dd6a5b8c7.debug...done.  
done.  
[New LWP 2773]  
[Thread debugging using libthread_db enabled]  
Using host libthread_db library "/lib/libthread_db.so.1".  
Core was generated by "/home/sbis/dev-socnet/socnet-service/service/sbis-daemon --name sbis-socnet-ser".  
Program terminated with signal 6, Aborted.  
#0 0xf7796430 in __kernel_vsyscall ()  
(gdb)
```

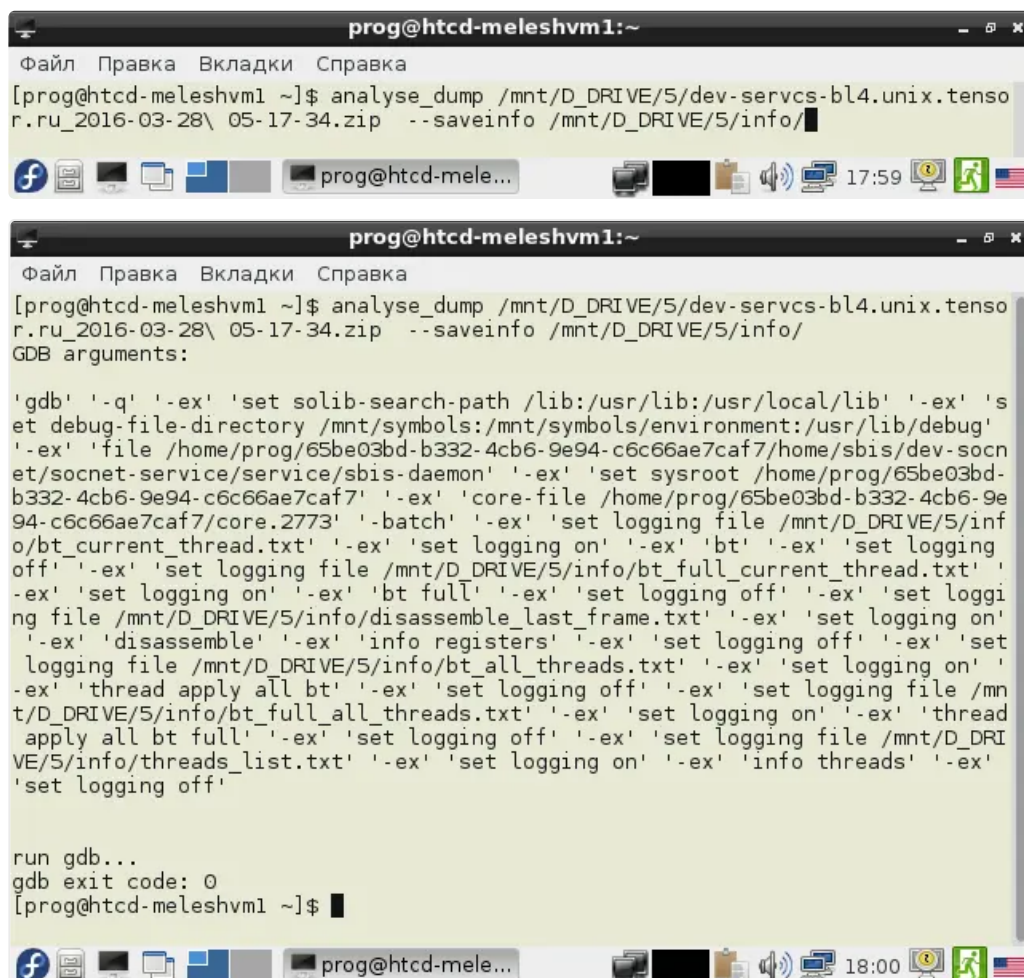
Неинтерактивный режим analyse_dump. Выгрузка информации из дампа в текстовые файлы

В неинтерактивном режиме утилита выгружает практически всю полезную информацию из дампа в текстовые файлы. Для запуска выгрузки нужно запустить утилиту с ключом "--saveinfo":

```
analyse_dump <путь к файлу дампа> --saveinfo <путь к папке, куда будет выгружена информация>
```

Сейчас ко всем заданиям по дампам, которые пишут тестировщики, прикладывается информация, выгруженная при помощи analyse_dump. В 99% случаев этой информации достаточно для анализа дампа и решения проблемы.

В примере ниже производится извлечение информации из дампа `/mnt/D_DRIVE/5/dev-servcs-bl4.unix.tensor.ru_2016-03-28\ 05-17-34.zip` в папку `/mnt/D_DRIVE/5/info`.



The image shows two screenshots of a terminal window titled 'prog@htcd-meleshvm1:~'. The first screenshot shows the command `analyse_dump /mnt/D_DRIVE/5/dev-servcs-bl4.unix.tensor.ru_2016-03-28\ 05-17-34.zip --saveinfo /mnt/D_DRIVE/5/info/` being entered. The second screenshot shows the output of the command, which is a list of GDB arguments for setting up a debugging environment. The arguments include setting the symbol path, debug file directory, sysroot, core file, logging file, and various logging options. The output ends with `run gdb...` and `gdb exit code: 0`.

```
prog@htcd-meleshvm1:~  
[prog@htcd-meleshvm1 ~]$ analyse_dump /mnt/D_DRIVE/5/dev-servcs-bl4.unix.tensor.ru_2016-03-28\ 05-17-34.zip --saveinfo /mnt/D_DRIVE/5/info/  
GDB arguments:  
'gdb' '-q' '-ex' 'set solib-search-path /lib:/usr/lib:/usr/local/lib' '-ex' 'set debug-file-directory /mnt/symbols:/mnt/symbols/environment:/usr/lib/debug' '-ex' 'file /home/prog/65be03bd-b332-4cb6-9e94-c6c66ae7caf7/home/sbis/dev-socnet/socnet-service/service/sbis-daemon' '-ex' 'set sysroot /home/prog/65be03bd-b332-4cb6-9e94-c6c66ae7caf7' '-ex' 'core-file /home/prog/65be03bd-b332-4cb6-9e94-c6c66ae7caf7/core.2773' '-batch' '-ex' 'set logging file /mnt/D_DRIVE/5/info/bt_current_thread.txt' '-ex' 'set logging on' '-ex' 'bt' '-ex' 'set logging off' '-ex' 'set logging file /mnt/D_DRIVE/5/info/bt_full_current_thread.txt' '-ex' 'set logging on' '-ex' 'bt full' '-ex' 'set logging off' '-ex' 'set logging file /mnt/D_DRIVE/5/info/disassemble_last_frame.txt' '-ex' 'set logging on' '-ex' 'disassemble' '-ex' 'info registers' '-ex' 'set logging off' '-ex' 'set logging file /mnt/D_DRIVE/5/info/bt_all_threads.txt' '-ex' 'set logging on' '-ex' 'thread apply all bt' '-ex' 'set logging off' '-ex' 'set logging file /mnt/D_DRIVE/5/info/bt_full_all_threads.txt' '-ex' 'set logging on' '-ex' 'thread apply all bt full' '-ex' 'set logging off' '-ex' 'set logging file /mnt/D_DRIVE/5/info/threads_list.txt' '-ex' 'set logging on' '-ex' 'info threads' '-ex' 'set logging off'  
  
run gdb...  
gdb exit code: 0  
[prog@htcd-meleshvm1 ~]$
```

В результате работы скрипта в указанной директории создадутся текстовые файлы с информацией, извлеченной из дампа:

```
prog@htcd-meleshvm1:~  
Файл Правка Вкладки Справка  
  
'gdb' '-q' '-ex' 'set solib-search-path /lib:/usr/lib:/usr/local/lib' '-ex' 'set  
debug-file-directory /mnt/symbols:/mnt/symbols/environment:/usr/lib/debug'  
'-ex' 'file /home/prog/65be03bd-b332-4cb6-9e94-c6c66ae7caf7/home/sbis/dev-socn  
et/socnet-service/service/sbis-daemon' '-ex' 'set sysroot /home/prog/65be03bd-  
b332-4cb6-9e94-c6c66ae7caf7' '-ex' 'core-file /home/prog/65be03bd-b332-4cb6-9e  
94-c6c66ae7caf7/core.2773' '-batch' '-ex' 'set logging file /mnt/D_DRIVE/5/info  
o/bt_current_thread.txt' '-ex' 'set logging on' '-ex' 'bt' '-ex' 'set logging  
off' '-ex' 'set logging file /mnt/D_DRIVE/5/info/bt_full_current_thread.txt' '-  
ex' 'set logging on' '-ex' 'bt full' '-ex' 'set logging off' '-ex' 'set loggi  
ng file /mnt/D_DRIVE/5/info/disassemble_last_frame.txt' '-ex' 'set logging on'  
'-ex' 'disassemble' '-ex' 'info registers' '-ex' 'set logging off' '-ex' 'set  
logging file /mnt/D_DRIVE/5/info/bt_all_threads.txt' '-ex' 'set logging on' '-  
ex' 'thread apply all bt' '-ex' 'set logging off' '-ex' 'set logging file /mnt  
/D_DRIVE/5/info/bt_full_all_threads.txt' '-ex' 'set logging on' '-ex' 'thread  
apply all bt full' '-ex' 'set logging off' '-ex' 'set logging file /mnt/D_DRI  
VE/5/info/threads_list.txt' '-ex' 'set logging on' '-ex' 'info threads' '-ex'  
'set logging off'  
  
run gdb...  
gdb exit code: 0  
[prog@htcd-meleshvm1 ~]$ ls /mnt/D_DRIVE/5/info  
bt_all_threads.txt      bt_full_current_thread.txt  threads_list.txt  
bt_current_thread.txt  disassemble_last_frame.txt  
bt_full_all_threads.txt  info.json  
[prog@htcd-meleshvm1 ~]$
```

Описание создаваемых скриптом текстовых файлов

Имя файла	Содержимое файла
bt_current_thread.txt	В данном файле отображен стек вызовов в потоке, в котором случилась проблема. Также отображены некоторые основные переменные (например, указатели this). Зачастую просмотра этого файла достаточно для выяснения причины падения в случае "наивных ошибок", например, при разыменовании нулевого указателя.
bt_full_current_thread.txt	В данном файле отображен стек вызовов в потоке, в котором случилась проблема. Дополнительно отображаются значения всех видимых отладчику локальных переменных и аргументов функций. Анализ этого файла позволяет выявить причину падения практически во всех случаях (за исключением падений из-за проблем многопоточности). В данном файле видны аргументы функций, поэтому легко проанализировать валидность обрабатываемых данных или вообще воспроизвести аварийную ситуацию локально, вызвав необходимые методы с нужными аргументами.
bt_all_threads.txt	В файле перечислены стеки вызовов всех потоков. Файл полезен при анализе дампов, возникших из-за проблем многопоточности.
bt_full_all_threads.txt	В файле перечислены стеки вызовов всех потоков с указанием значений всех локальных переменных и аргументов вызова функций, видимых отладчиком. Файл полезен при анализе дампов, возникших из-за проблем многопоточности.
threads_list.txt	Список всех потоков с указанием их идентификаторов. Файл полезен при анализе дампов, возникших из-за проблем многопоточности. Идентификаторы потоков нужны для поиска логов по нужному потоку (в сервисе логов).
disassemble_last_frame.txt	В файле представлен ассемблерный листинг последнего выполненного фрейма в потоке, в котором возникла проблема. Так же в файле приведена информация о состоянии регистров на момент падения процесса.

	Данный файл полезен только для людей, владеющих ассемблером. Ассемблерный листинг последнего фрейма очень часто помогает разобраться в проблемах, которые не удалось решить другими способами.
info.json	В файле содержится общая информация о процессе и об его окружении на момент падения.

Пример процесса анализа дампа

Для наглядности возьмем [типовое задание](#) по дампам и произведем его анализ. К заданию приложены [вышеперечисленные файлы](#). В большинстве случаев их будет достаточно для решения проблемы, поэтому даже не придется запускать виртуальную Linux-машину.

1. Смотрим файл `bt_current_thread.txt`.

```

1 #0 operator-> (this=0x4) at core-common/sbis-
  lib/implementation/record/recordset.cpp:4671
2 #1 sbis::RecordSet::Size (this=0x0) at core-common/sbis-
  lib/implementation/record/recordset.cpp:2478
3 #2 0xf3c03d95 in operator() (__closure=0xf1c630b0) at
  implementation/methods/rpc_methods.cpp:159
4 #3 0xf3c131b3 in operator() (this=) at /usr/lib/gcc/i686-redhat-
  linux/4.7.2/../../../../include/c++/4.7.2/functional:2311
5 #4
  sbis::smartbuffer::ProcessErrors(sbis::smartbuffer::ServiceErrorCode
  const&, std::function, std::function) (error_code=@0xf13fd31c:
  sbis::smartbuffer::ercERROR_ADD_FILES, function=, finallyfunc=...) at
  implementation/serializable_exceptions.cpp:21
6 #5 0xf3bf0dae in sbis::smartbuffer::SBISBuffer_AddFiles (result=...,
  IsPersistent=@0xf13fd3f7: false, Content=...) at
  implementation/methods/rpc_methods.cpp:248
7 #6 0xf3c077f9 in sbis::detail::ParametersDeserializer<2u, 2u, false,
  void ()(std::basic_string, std::allocator >&, bool&,
  std::shared_ptr&), std::basic_string, std::allocator > >::Expand >
  (void ()(std::basic_string, std::allocator >&, bool&,
  std::shared_ptr&), sbis::IContext&, sbis::IObjectReader&,
  sbis::IReturnWriter&, std::array, std::allocator >, 2u> const&,
  std::basic_string, std::allocator >&, bool&, std::shared_ptr&)
  (func=0xf3bf0d30, std::allocator >&, bool&, std::shared_ptr&)>,
  writer=..., result=...) at
  /home/sbis/mount_disk_sdk/SDK/linux/3.7.3.150/106/include/sbis-
  lib/func_list/func_registrator_utils_lin.hpp:121

```

2. Сразу обращаем внимание на то, что в 1-ом фрейме указатель `this` является нулевым. Это значит, что во втором фрейме было произведено разыменоване нулевого указателя и вызов метода. То есть выполнен код, подобный следующему:

```

1 SomeClass* ptr = nullptr;
2 ptr->SomeMethod(); // в фрейме SomeClass::SomeMethod будет this ==
  0x0

```

3. Смотрим код "implementation/methods/rpc_methods.cpp:159":

```

1 RPC_FUNC_2(
2     L"SBISBuffer.AddFiles",
3     String,
4     SBISBuffer_AddFiles,
5     Bool,
6     L"IsPersistent",
7     IsPersistent,
8     RecordSetSPtr,

```

```

9      L"Content",
10     Content
11  )
12  {
13      ProcessErrors(ercERROR_ADD_FILES, [&]
14      {
15          if (Content->Size() > 0)
16          ...

```

4. В файле `bt_full_current_thread.txt` видно, что `Content` является нулевым указателем, что подтверждает гипотезу:

```

1  Content = @0xf13fd398: {<std::__shared_ptr<sbis::RecordSet,
    (__gnu_cxx::__Lock_policy)2>> = {_M_ptr = 0x0, _M_refcount = {_M_pi =
    0x0}}, <No data fields>}

```

В данном коде производится разыменование указателя, полученного в аргументах метода, без его предварительной проверки. Падение произошло из-за того, что в метод пришел нулевой указатель и он был разыменован.

Способ исправления проблемы зависит от логики приложения. Тут могут быть два варианта:

1. Скорее всего для этого метода ситуация, когда `Content` нулевой, не является корректной. Поэтому в таком случае правильно будет изменить тип аргумента и принимать `RecordSet`, а не `RecordSetSPtr`. В таком случае, если снаружи передано `null` в качестве аргумента, платформенные механизмы вызова сами проверят аргумент и выдадут корректную ошибку, если он окажется пустым.
2. Если ситуация, когда `Content` нулевой, является для метода корректной, то нужно реализовать отдельную ветку кода на этот случай.

Таким образом, даже не переключаясь из окна браузера, можно проанализировать дампы и разработать способы решения проблемы.

Открытие дампов при помощи отладчика GDB

Справочную информацию по отладчику можно найти на [официальном сайте](#).

Шаг 1: Распакуйте архив с дампом в домашнюю директорию

Например, домашняя директория:

```
/home/local/TENSOR-CORP/aa.bykanov/
```

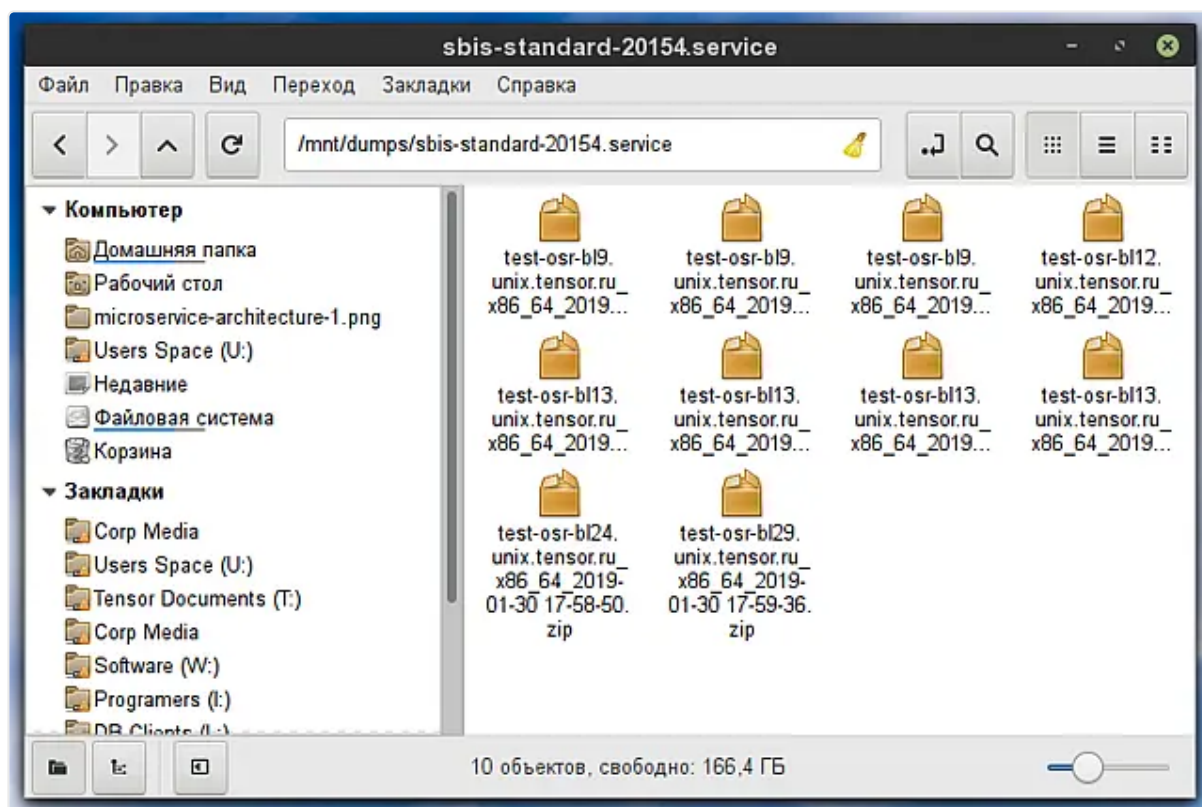
тогда содержимое архива дампа распакуйте в директорию `dumps`:

```
/home/local/TENSOR-CORP/aa.bykanov/dumps/
```

О получении дампов "боевого" приложения можно прочитать [здесь](#), а для локального стенда — [здесь](#).

Например, дамп приложения был скачан с сетевого диска:

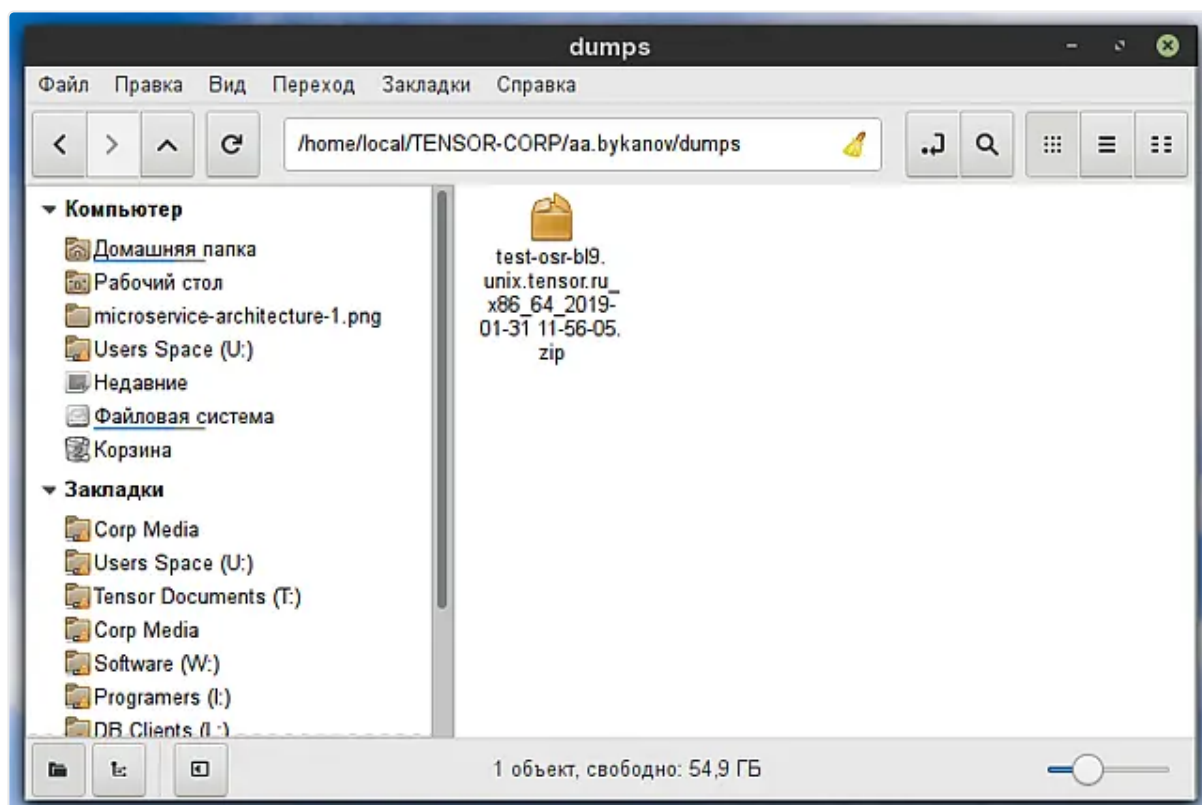
```
/mnt/dumps/sbis-standard-20154.service/test-osr-
bl9.unix.tensor.ru_x86_64_2019-01-31_11-56-05.zip
```

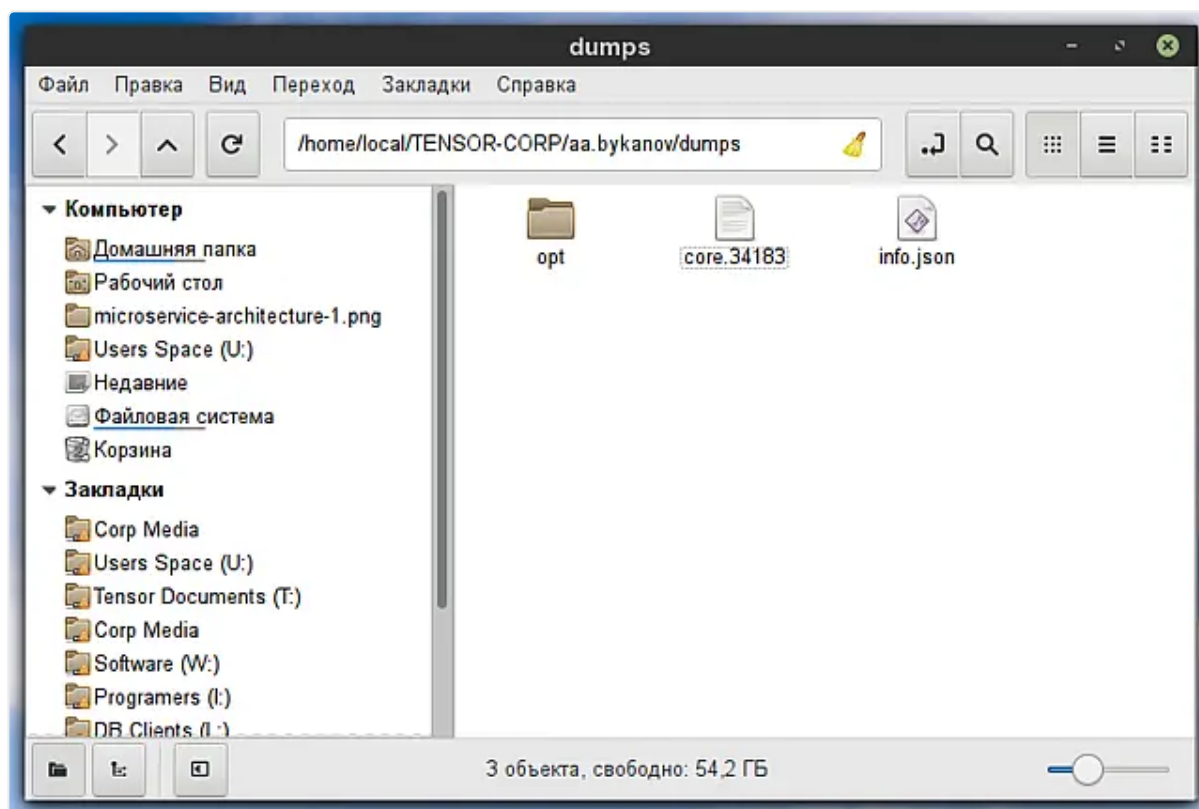


Его содержимое нужно распаковать в директорию `/home/local/TENSOR-CORP/aa.bykanov/dumps/`, где будет сформирована следующая файловая структура:

```
/home/local/TENSOR-CORP/aa.bykanov/dumps/
```

```
├── opt
│   └── sbis
│       └── standard1
│           └── online
│               └── ...
├── core.34183
└── info.json
```





Шаг 2 (опционально): Примонтируйте раздел сетевого диска, на котором расположены отладочные символы

Если вы получили рабочий компьютер с предварительно установленным дистрибутивом CentOS 7.5, тогда раздел сетевого диска уже будет смонтирован, и данный шаг можно пропустить. Это можно проверить, если в адресной строке файлового менеджера Nemo открыть:

```
/mnt/symbols
```

Также такой раздел монтируется автоматически при установке метапакета [sbis-development-tools](#).

Чтобы смонтировать раздел вручную, откройте терминал и выполните следующую команду:

```
sudo mount -v dev-symbols.unix.tensor.ru:/srv/data/Symbols_nix_R0  
/mnt/symbols -t nfs -o ro,async,soft,noatime
```

При успешном монтировании раздела в терминале будет такое сообщение:

```
1 mount.nfs: timeout set for Thu Jan 31 16:12:35 2019  
2 mount.nfs: trying text-based options  
  'soft,vers=4.1,addr=10.76.162.27,clientaddr=10.76.188.191'
```

Если диск уже смонтирован, тогда в терминале будет такое сообщение:

```
1 mount.nfs: timeout set for Wed Jan 30 13:56:33 2019  
2 mount.nfs: trying text-based options  
  'soft,vers=4.1,addr=10.76.162.27,clientaddr=10.76.188.191'  
3 mount.nfs: mount(2): Device or resource busy  
4 mount.nfs: /mnt/symbols is busy or already mounte
```

Шаг 3: Запустите отладчик GDB

Для отладки требуется GDB версии не ниже 8. Проверить версию можно командой:


```
gdb --version
```

Если у вас в системе основная версия GDB ниже, то можно использовать другой GDB из альтернативной "коллекции ПО" через механизм "RedHat Software Collections" в CentOS.

Для этого нужно убедиться, что эта "коллекция" установлена, либо установить ее через:

```
sudo yum install devtoolset-8
```

Далее, чтобы открыть сеанс консоли bash, используя эту коллекцию, нужно набрать:

```
scl enable devtoolset-8 bash
```

Запустится новый сеанс консоли, в котором будет активирована эта коллекция. Чтобы проверить, что результат достигнут, можно снова набрать:

```
gdb --version
```

и убедиться, что в выводе будет более новая версия GDB. Использовать scl enable нужно каждый раз при запуске нового сеанса консоли, но устанавливать пакет devtoolset-8 — только один раз.

Из домашней директории откройте [терминал](#) и выполните команду запуска отладчика:

```
gdb
```

Теперь можно взаимодействовать с отладчиком посредством команд.

Следующие шаги написаны исходя из того, что отладчик GDB запущен из домашней директории.

Шаг 4: Установите в GDB путь к серверу с отладочными символами

Для этого используйте команду [debug-file-directory](#).

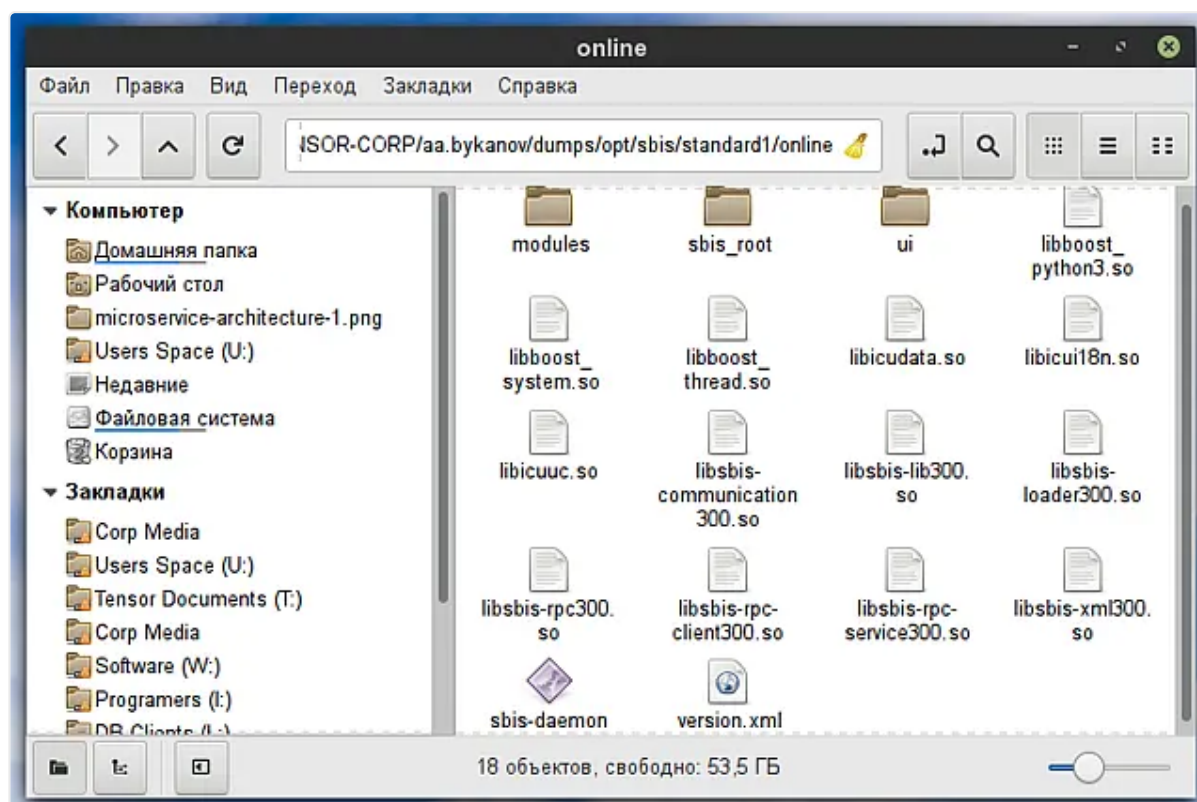
Пример выполнения команды:

```
(gdb) set debug-file-directory  
/usr/lib/debug/:/mnt/symbols:/mnt/symbols/environment:/mnt/symbols/19.110/Platformforma/linux_x86_64/21:/mnt/symbols/19.110/online-inside/728.137
```

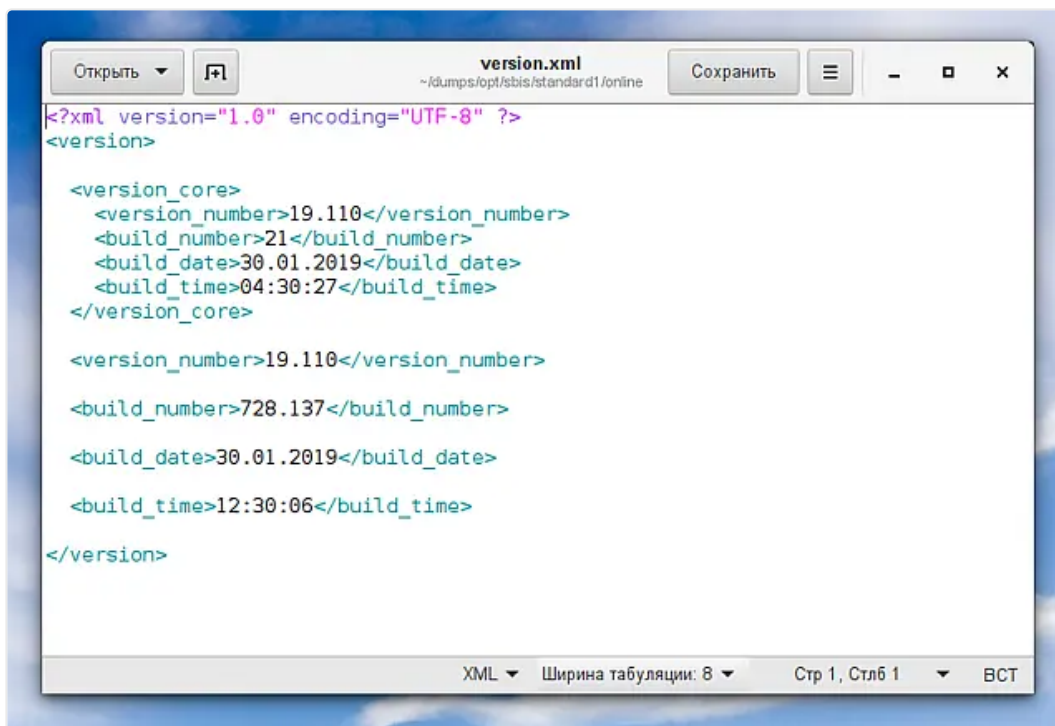
```
aa.bykanov@usd-bykanov1:~  
Файл Правка Вид Поиск Терминал Справка  
[aa.bykanov@usd-bykanov1 ~]$ gdb  
GNU gdb (GDB) Red Hat Enterprise Linux 7.6.1-110.el7  
Copyright (C) 2013 Free Software Foundation, Inc.  
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>  
This is free software: you are free to change and redistribute it.  
There is NO WARRANTY, to the extent permitted by law. Type "show copying"  
and "show warranty" for details.  
This GDB was configured as "x86_64-redhat-linux-gnu".  
For bug reporting instructions, please see:  
<http://www.gnu.org/software/gdb/bugs/>.  
(gdb) set debug-file-directory /usr/lib/debug:/mnt/symbols:/mnt/symbols/environ  
ment:/mnt/symbols/19.110/Platforma/linux_x86_64/21:/mnt/symbols/19.110/online-in  
side/728.137  
(gdb) □
```

Здесь 19.110/Platforma/linux_x86_64/21 и 19.110/online-inside/728.137 являются вариативными частями. Чтобы определить путь к отладочным символам для вашего дампа, сделайте следующее:

1. Откройте в директории дампа файл **opt/sbis/standard1/online/version.xml**.



2. В корне секции **version** указана версия и номер сборки (**build_number**) приложения, а в секции **version_core** — версия и билд Платформы.



Отладочные символы расположены на сетевом диске `/mnt/symbols/`, монтирование которого рассмотрено на [шаге № 2](#).

Шаг 5: Установите в GDB путь до сторонних системных библиотек

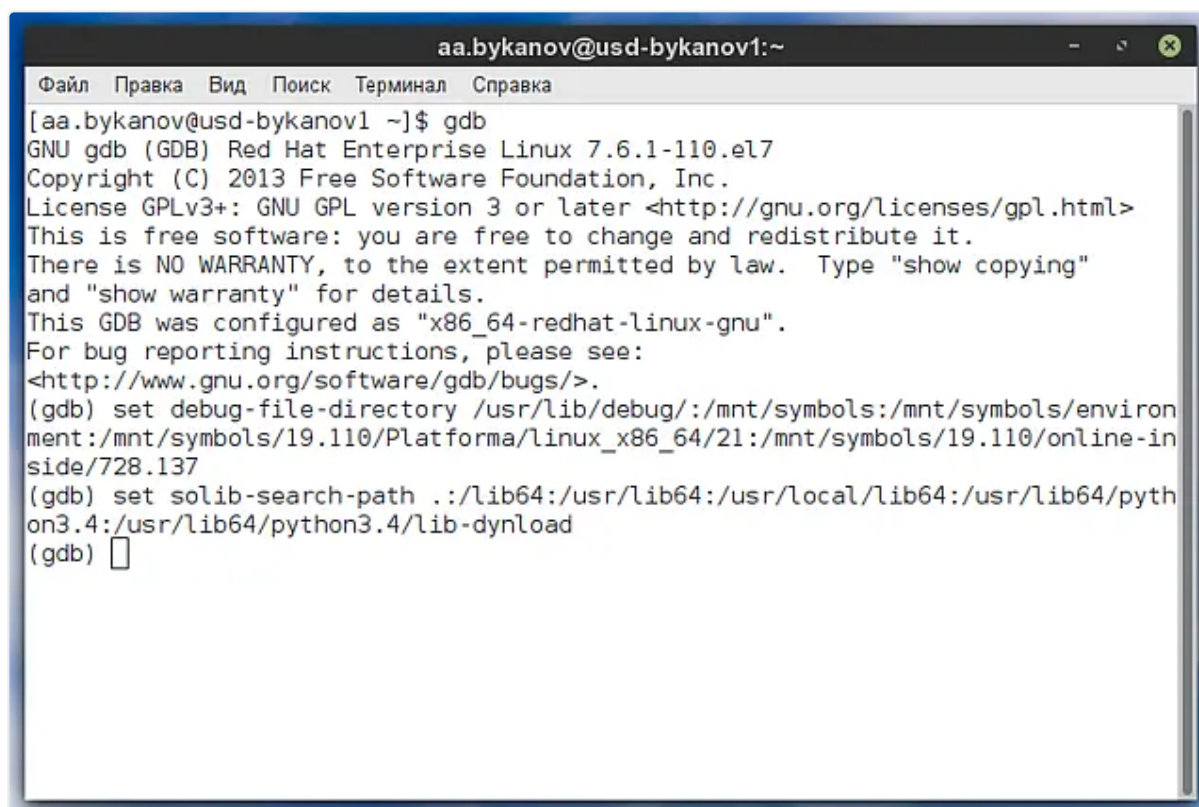
Для этого используйте команду `solib-search-path`. Следующий пример описывает подключение основных системных библиотек. Выполнение такой команды подходит для большинства случаев.

```
(gdb) set solib-search-path
./lib64:/usr/lib64:/usr/local/lib64:/usr/lib64/python3.4:/usr/lib64/python3.4/lib-dynload
```



Примечание

Успешно выполненная команда не выведет в консоль никаких сообщений.



```
aa.bykanov@usd-bykanov1:~  
Файл Правка Вид Поиск Терминал Справка  
[aa.bykanov@usd-bykanov1 ~]$ gdb  
GNU gdb (GDB) Red Hat Enterprise Linux 7.6.1-110.el7  
Copyright (C) 2013 Free Software Foundation, Inc.  
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>  
This is free software: you are free to change and redistribute it.  
There is NO WARRANTY, to the extent permitted by law. Type "show copying"  
and "show warranty" for details.  
This GDB was configured as "x86_64-redhat-linux-gnu".  
For bug reporting instructions, please see:  
<http://www.gnu.org/software/gdb/bugs/>.  
(gdb) set debug-file-directory /usr/lib/debug/:/mnt/symbols:/mnt/symbols/environ  
ment:/mnt/symbols/19.110/Platforma/linux_x86_64/21:/mnt/symbols/19.110/online-in  
side/728.137  
(gdb) set solib-search-path ./lib64:/usr/lib64:/usr/local/lib64:/usr/lib64/pyth  
on3.4:/usr/lib64/python3.4/lib-dynload  
(gdb) █
```

Шаг 6: Установите в GDB относительный путь к директории, в которую распакован архив с дампом

Для этого применяется команда **solib-absolute-prefix**. Например, если дампы распакованы в директорию:

```
/home/local/TENSOR-CORP/aa.bykanov/dumps/
```

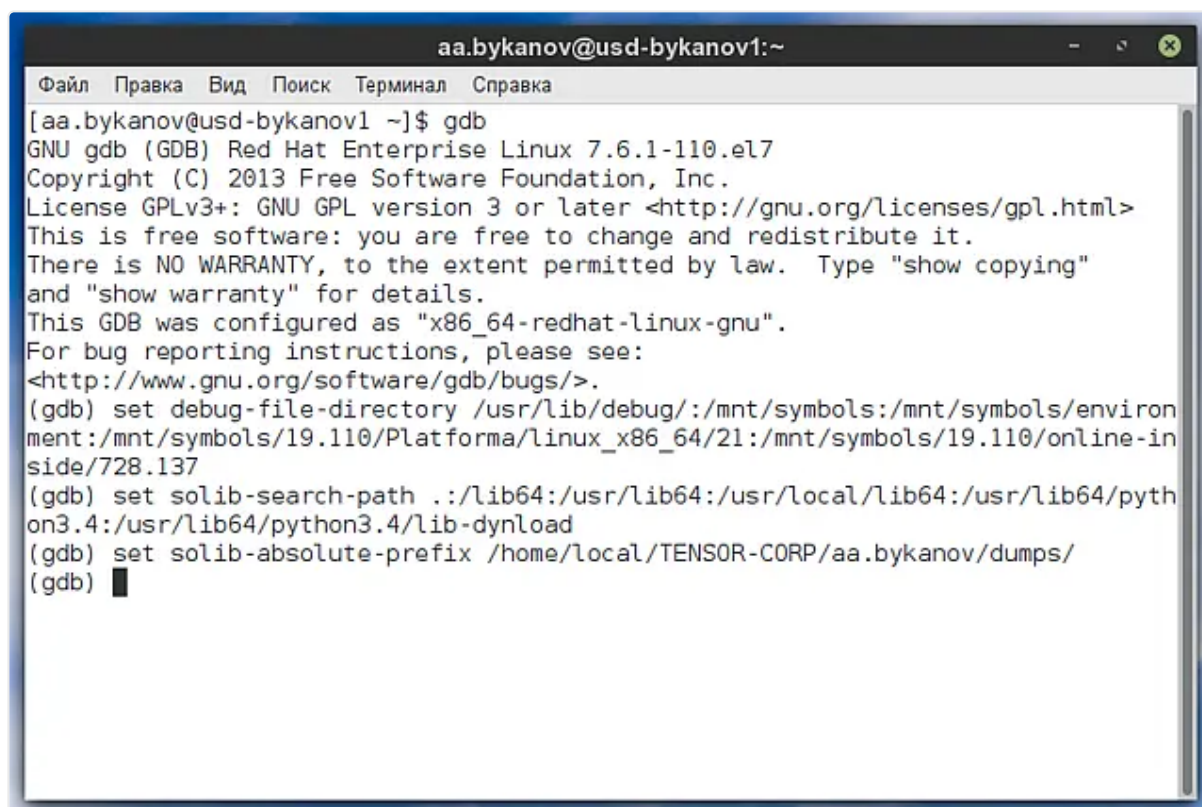
тогда сама команда будет такой:

```
(gdb) set solib-absolute-prefix /home/local/TENSOR-CORP/aa.bykanov/dumps/
```



Примечание

Успешно выполненная команда не выведет в консоль никаких сообщений.



```
aa.bykanov@usd-bykanov1:~  
Файл Правка Вид Поиск Терминал Справка  
[aa.bykanov@usd-bykanov1 ~]$ gdb  
GNU gdb (GDB) Red Hat Enterprise Linux 7.6.1-110.el7  
Copyright (C) 2013 Free Software Foundation, Inc.  
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>  
This is free software: you are free to change and redistribute it.  
There is NO WARRANTY, to the extent permitted by law. Type "show copying"  
and "show warranty" for details.  
This GDB was configured as "x86_64-redhat-linux-gnu".  
For bug reporting instructions, please see:  
<http://www.gnu.org/software/gdb/bugs/>.  
(gdb) set debug-file-directory /usr/lib/debug/:/mnt/symbols:/mnt/symbols/environ  
ment:/mnt/symbols/19.110/Platforma/linux_x86_64/21:/mnt/symbols/19.110/online-in  
side/728.137  
(gdb) set solib-search-path ./lib64:/usr/lib64:/usr/local/lib64:/usr/lib64/pyth  
on3.4:/usr/lib64/python3.4/lib-dynload  
(gdb) set solib-absolute-prefix /home/local/TENSOR-CORP/aa.bykanov/dumps/  
(gdb) █
```

Шаг 7: Оптимизируйте в GDB читаемость информации, выводимой отладчиком в терминал

Для этого выполните команды:

- 1 (gdb) set pagination off
- 2 (gdb) set print elements 0

Команда **pagination off** отключает вывод данных на нескольких страницах. Т.е. будут видны сразу все фреймы.



Примечание

Успешно выполненная команда не выведет в консоль никаких сообщений.


```
aa.bykanov@usd-bykanov1:~
Файл  Правка  Вид  Поиск  Терминал  Справка
[aa.bykanov@usd-bykanov1 ~]$ gdb
GNU gdb (GDB) Red Hat Enterprise Linux 7.6.1-110.el7
Copyright (C) 2013 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.  Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
(gdb) set debug-file-directory /usr/lib/debug/:mnt/symbols:/mnt/symbols/environ
ment:/mnt/symbols/19.110/Platforma/linux_x86_64/21:/mnt/symbols/19.110/online-in
side/728.137
(gdb) set solib-search-path ./lib64:/usr/lib64:/usr/local/lib64:/usr/lib64/pyth
on3.4:/usr/lib64/python3.4/lib-dynload
(gdb) set solib-absolute-prefix /home/local/TENSOR-CORP/aa.bykanov/dumps/
(gdb) set pagination off
(gdb) set print elements 0
(gdb) █
```

Шаг 8: Загрузите в GDB бинарные файлы

Для этого применяется команда **file**. Например, если отладчик запущен из домашней директории, а дамп распакован в директорию `/home/local/TENSOR-CORP/aa.bykanov/dumps/`, тогда команда будет такой:

```
(gdb) file ./dumps/opt/sbis/standard1/online/sbis-daemon
```

Здесь часть пути — `standard1/online/` — является вариативной и меняется для разных дампов. Но в общем случае здесь указывается путь до файла **sbis-daemon**.

```
aa.bykanov@usd-bykanov1:~
Файл  Правка  Вид  Поиск  Терминал  Справка
[aa.bykanov@usd-bykanov1 ~]$ gdb
GNU gdb (GDB) Red Hat Enterprise Linux 7.6.1-110.el7
Copyright (C) 2013 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.  Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
(gdb) set debug-file-directory /usr/lib/debug/:mnt/symbols:/mnt/symbols/environ
ment:/mnt/symbols/19.110/Platforma/linux_x86_64/21:/mnt/symbols/19.110/online-in
side/728.137
(gdb) set solib-search-path ./lib64:/usr/lib64:/usr/local/lib64:/usr/lib64/pyth
on3.4:/usr/lib64/python3.4/lib-dynload
(gdb) set solib-absolute-prefix /home/local/TENSOR-CORP/aa.bykanov/dumps/
(gdb) set pagination off
(gdb) set print elements 0
(gdb) file ./dumps/opt/sbis/standard1/online/sbis-daemon
Reading symbols from /home/local/TENSOR-CORP/aa.bykanov/dumps/opt/sbis/standard1
/online/sbis-daemon...Reading symbols from /mnt/symbols/19.110/Platforma/linux_x
86_64/21/.build-id/f3/45a9d3f3e80fda9daaed90dd163aeffa125178.debug...done.
done.
(gdb) █
```

Шаг 9: Загрузите в GDB core-файл

Для этого применяется команда **core-file**.

Этот файл поставляется в архиве с дампом. Например, в рамках данного руководства дамп распакован по адресу `/home/local/TENSOR-CORP/aa.bykanov/dumps/`. Тогда подключение core-файл будет выполнено следующей командой:

```
(gdb) core-file ./dumps/core.34183
```

Здесь core.34183 является вариативной частью и отличается для каждого дампа.

Шаг 10: Анализ дампа

Далее можно производить анализ дампа при помощи команд GDB, полный список которых приведён на [официальном сайте](#). Ниже приведен список команд, используемых для анализа:

- просмотр стека:

```
(gdb) bt
```

- просмотр тредов:

```
(gdb) info thread
```

- переключение на конкретный тред и просмотр его стека:

```
1 | (gdb) thread 14
2 | (gdb) bt
```

Ошибки при отладке через GDB и способы их решения

The debug information does not match

Примерный текст ошибки:

```
warning: the debug information found in "[путь к файлу]" does not match "[путь к файлу]" (CRC mismatch)
```

```
aa.bykanev@usd-bykanev:~$  
Файл Правка Вид Поиск Терминал Справка  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/_unicodedata.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/_unicodedata.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/_ctypes.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/_ctypes.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/_struct.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/_struct.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/_heapq.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/_heapq.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/time.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/time.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/_posixsubprocess.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/_posixsubprocess.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/select.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/select.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/math.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/math.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/grp.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/grp.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/_bz2.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/_bz2.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/_hashlib.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/_hashlib.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/_random.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/_random.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/_datetime.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/_datetime.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/_decimal.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/_decimal.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/_json.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/_json.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/binascii.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/binascii.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/_zlib.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/_zlib.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/_lzma.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/_lzma.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/array.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/array.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/_mmap.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/_mmap.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/_elementtree.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/_elementtree.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/pyexpat.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/pyexpat.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/_opcode.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/_opcode.cpython-34m.so" (CRC mismatch).  
warning: the debug information found in "/usr/lib/debug//usr/lib64/python3.4/lib-dynload/_bisect.cpython-34m.so.debug" does not match "/usr/lib64/python3.4/lib-dynload/_bisect.cpython-34m.so" (CRC mismatch).  
Core was generated by "/opt/ibis/standard/online/ibis-daemon --pool 0 --queue_name Beff635af6a64ca192".  
Program terminated with signal 11, Segmentation fault.  
core dumped
```

Сообщение свидетельствует о том, что версии пакетов python34 и python34-debuginfo отличаются. Для решения ошибки следует обратиться к сотрудникам отдела [Техподдержка ГО](#).